

MATLAB 프로그래밍 14주차 과제

Spring 2026

안내사항

- 하나의 Live Script 파일로 (***.mlx) 로 작성하고 각 문제는 섹션으로 구분할 것. 하나의 파일이니 파일명에 문제번호는 삭제.
- 과제 메뉴에 업로드는 mlx 파일과 pdf 파일(Export(내보내기) 기능을 활용) 2개를 업로드 할 것.
- mlx 파일 안에 여러 변수가 중복될 수 있으니 문제마다 코드 시작 시 clear, clc 를 적절히 사용할 것.
- 댓글 창에 본인의 코드를 github에 저장하고 링크를 제출할 것.
- 문제에서 지정한 변수명과 그래프 제목, 축 이름, legend를 정확히 사용할 것.
- 난수를 사용하는 문제에서는 문제에서 seed가 지정된 경우 반드시 해당 seed를 사용할 것.

1. 한 물류센터에서는 1시간 동안 들어오는 배송 요청 건수를 시뮬레이션하려고 한다. 평소 관측 결과, 1시간당 평균 배송 요청 건수는 18건이라고 한다. 하루를 12시간 운영한다고 가정하고, 30일 동안의 배송 요청 건수를 난수로 생성하여 분석하시오.

- (a) 난수 결과가 재현되도록 `rng(14)`를 설정하시오.
- (b) 평균이 18인 포아송분포 난수를 이용하여 30×12 크기의 행렬 `request`를 생성하시오. 행은 날짜, 열은 시간대를 의미하도록 하시오.
- (c) 각 날짜별 총 요청 건수를 계산하여 30×1 열벡터 `dailyTotal`에 저장하시오.
- (d) 하루 총 요청 건수가 240건 이상인 날짜를 `busyDay`라는 logical vector로 저장하시오.
- (e) 혼잡일의 개수를 `numBusyDay`에 저장하시오.
- (f) 전체 30일 동안의 시간대별 평균 요청 건수를 `hourlyMean`에 저장하시오.
- (g) `dailyTotal`은 선 그래프로 나타내고, 혼잡일은 같은 그래프 위에 marker로 표시하시오.
- (h) x축 label은 'Day', y축 label은 'Total Requests', 제목은 'Daily Delivery Request Simulation'으로 설정하시오.

2. 한 데이터센터의 서버는 작업 요청이 무작위로 도착하고, 각 작업의 처리 시간도 무작위로 달라진다. 작업 도착 간격은 평균 1.2초의 지수분포를 따르고, 작업 처리 시간은 평균 0.9초의 지수분포를 따른다고 하자. 서버는 한 번에 하나의 작업만 처리할 수 있다.

- (a) 작업 수를 $n = 3000$ 으로 설정하시오.
- (b) 도착 간격을 `interArrival`이라는 $1 \times n$ 벡터로 생성하시오.
- (c) `cumsum`을 이용하여 실제 도착 시각 벡터 `arrival`을 생성하시오.
- (d) 처리 시간을 `service`라는 $1 \times n$ 벡터로 생성하시오.

- (e) 각 작업의 서비스 시작 시각 `startTime`과 종료 시각 `finishTime`을 for문으로 계산하시오.
 - (f) 대기시간 `waitingTime = startTime - arrival`을 계산하시오.
 - (g) 평균 대기시간, 최대 대기시간, 대기시간이 5초 이상인 작업의 비율을 각각 `meanWait`, `maxWait`, `longWaitRatio`에 저장하시오.
 - (h) `waitingTime`의 histogram을 40개 구간으로 나타내시오.
 - (i) 그래프 제목은 'Waiting Time Distribution in a Data Center Queue'로 설정하시오.
3. 한 전자제품은 4개의 핵심 부품으로 구성되어 있으며, 4개 부품이 모두 정상 작동해야 제품이 정상 작동한다. 각 부품의 정상 작동 확률은 각각 0.97, 0.95, 0.96, 0.92이다. 100000개의 제품에 대해 제품 정상 작동 여부를 시뮬레이션하시오.
- (a) 제품 수를 `n = 100000`으로 설정하시오.
 - (b) 각 부품의 정상 작동 여부를 `c1`, `c2`, `c3`, `c4`라는 logical vector로 생성하시오.
 - (c) 4개 부품이 모두 정상 작동하는 경우만 제품이 정상 작동한다고 정의하고, 결과를 `systemWorking`에 저장하시오.
 - (d) 시뮬레이션으로 추정한 제품 신뢰도를 `reliabilityEst`에 저장하시오.
 - (e) 이론적 신뢰도 $0.97 \times 0.95 \times 0.96 \times 0.92$ 를 계산하여 `reliabilityTrue`에 저장하시오.
 - (f) `abs(reliabilityEst - reliabilityTrue)`를 계산하여 `absError`에 저장하시오.
 - (g) 각 부품의 고장 수와 전체 시스템 고장 수를 하나의 벡터 `failCounts`에 저장하시오.
 - (h) `failCounts`를 bar graph로 나타내고, x축 눈금 이름을 'C1', 'C2', 'C3', 'C4', 'System'으로 지정하시오.
4. 한 배터리 제조사는 배터리 사용 시간이 평균 9.5시간, 표준편차 1.2시간인 정규분포를 따른다고 가정한다. 배터리 사용 시간이 8시간 미만이면 “부족”, 8시간 이상 11시간 미만이면 “정상”, 11시간 이상이면 “우수”로 분류한다. 50000개 배터리를 시뮬레이션하시오.
- (a) `batteryLife`라는 1×50000 벡터를 생성하시오.
 - (b) 사용 시간이 8시간 미만인 배터리의 logical vector를 `lowBattery`에 저장하시오.
 - (c) 사용 시간이 8시간 이상 11시간 미만인 배터리의 logical vector를 `normalBattery`에 저장하시오.
 - (d) 사용 시간이 11시간 이상인 배터리의 logical vector를 `excellentBattery`에 저장하시오.
 - (e) 세 등급의 개수를 각각 계산하여 `gradeCounts`라는 1×3 벡터에 저장하시오.
 - (f) 세 등급의 비율을 `gradeRatio`라는 1×3 벡터에 저장하시오.
 - (g) 배터리 수명 histogram을 50개 구간으로 나타내고, $x = 8$, $x = 11$ 위치에 기준선을 추가하시오.
 - (h) 별도의 figure에 `gradeCounts`를 bar graph로 나타내시오.
5. 한 보험회사는 고객 20000명에 대해 보험 청구 여부와 청구액을 시뮬레이션하려고 한다. 고객별 보험 청구 확률은 0.06이다. 청구가 발생한 경우 청구액은 평균 800, 표준편차 250인

정규분포를 따른다고 가정한다. 단, 정규분포로 생성된 청구액이 음수이면 해당 값을 0으로 바꾼 뒤 계산하시오.

- (a) 고객 수를 $n = 20000$ 으로 설정하시오.
 - (b) 고객별 청구 여부를 `hasClaim`이라는 logical vector로 생성하시오.
 - (c) 평균 800, 표준편차 250인 정규분포 난수를 이용하여 `claimAmount`를 생성하시오.
 - (d) `claimAmount` 중 음수인 값을 모두 0으로 바꾸시오.
 - (e) 실제 지급액을 `claimPayment = hasClaim .* claimAmount`로 계산하시오.
 - (f) 총 지급액 `totalPayment`, 고객 1인당 평균 지급액 `meanPayment`, 청구가 발생한 고객에 대한 평균 지급액 `meanClaimOnly`를 계산하시오.
 - (g) 지급액이 1200 이상인 고객 청구 고객 수를 `numLargeClaim`에 저장하시오.
 - (h) `claimPayment` 전체 histogram과 `claimPayment > 0`인 값만의 histogram을 서로 다른 figure에 나타내시오.
6. 한 입자는 매 단계마다 왼쪽 또는 오른쪽으로 1칸씩 이동한다. 단, 오른쪽으로 이동할 확률은 0.56이고 왼쪽으로 이동할 확률은 0.44이다. 총 1000단계의 random walk를 100개 생성하여 경로의 퍼짐과 최종 위치를 분석하시오.
- (a) 단계 수를 $n = 1000$, 경로 수를 $m = 100$ 으로 설정하시오.
 - (b) 오른쪽 이동 확률 0.56을 반영하여 $n \times m$ 크기의 이동 행렬 `steps`를 생성하시오. 오른쪽 이동은 +1, 왼쪽 이동은 -1로 저장하시오.
 - (c) `cumsum`을 이용하여 각 경로의 누적 위치 행렬 `walks`를 생성하시오.
 - (d) 100개 경로를 하나의 figure에 모두 그리시오.
 - (e) 각 경로의 최종 위치를 `finalPosition`이라는 $1 \times m$ 벡터로 저장하시오.
 - (f) 최종 위치의 평균과 표준편차를 각각 `meanFinal`, `stdFinal`에 저장하시오.
 - (g) 최종 위치가 100 이상인 경로의 비율을 `rightDriftRatio`에 저장하시오.
 - (h) 별도의 figure에 `finalPosition`의 histogram을 30개 구간으로 나타내시오.

7. 한 센서가 시간에 따라 다음 참 신호를 측정한다고 하자.

$$s(t) = 2 \sin(0.4t) + 0.7 \cos(1.5t).$$

센서는 정규분포 잡음을 포함하여 신호를 측정하며, 전체 측정값 중 일부가 누락된다. 누락된 데이터를 선형보간과 스플라인 보간으로 각각 복원하고 성능을 비교하시오.

- (a) 난수 결과가 재현되도록 `rng(7)`을 설정하시오.
- (b) `t = linspace(0,30,300)`으로 시간 벡터를 생성하시오.
- (c) 참 신호를 `trueSignal`에 저장하시오.
- (d) 표준편차 0.25의 정규분포 잡음을 추가하여 관측 신호 `obsSignal`을 생성하시오.
- (e) `randperm`을 사용하여 전체 위치 중 45개를 선택하고, 해당 위치의 `obsSignal` 값을 NaN으로 바꾸시오.

- (f) 결측치가 아닌 위치를 logical vector `valid`에 저장하시오.
- (g) 결측치가 아닌 데이터만 사용하여 전체 시간 벡터 `t`에 대해 선형보간 결과 `recLinear`를 생성하시오.
- (h) 같은 방식으로 스플라인 보간 결과 `recSpline`을 생성하시오.
- (i) 참 신호와 `recLinear`, `recSpline` 사이의 RMSE를 각각 `rmseLinear`, `rmseSpline`에 저장하시오.
- (j) 참 신호, 결측치가 포함된 관측값, 선형보간 결과, 스플라인 보간 결과를 하나의 그래프에 나타내고 legend를 추가하시오.

8. 한 기상 관측소는 3시간 간격으로 하루 동안의 풍속을 측정하였다. 다음 데이터는 관측 시각과 풍속이다.

Time	0	3	6	9	12	15	18	21	24
WindSpeed	3.1	3.8	5.4	7.2	8.0	7.1	5.6	4.2	3.5

1시간 간격의 풍속을 선형보간과 스플라인 보간으로 추정하고 두 결과의 차이를 분석하시오.

- (a) 관측 시각을 `timeObs`, 관측 풍속을 `windObs`에 저장하시오.
- (b) 0부터 24까지 1시간 간격의 벡터 `timeHourly`를 생성하시오.
- (c) `interp1`을 이용하여 선형보간 결과를 `windLinear`에 저장하시오.
- (d) `interp1`을 이용하여 스플라인 보간 결과를 `windSpline`에 저장하시오.
- (e) 두 보간 결과의 차이 `diffWind = windSpline - windLinear`를 계산하시오.
- (f) `abs(diffWind)`의 최대값을 `maxDiff`에 저장하시오.
- (g) 관측 데이터, 선형보간 결과, 스플라인 보간 결과를 하나의 그래프에 나타내시오.
- (h) 별도의 figure에 `diffWind`를 선 그래프로 나타내시오.

9. 어떤 지형의 높이를 5×5 격자점에서만 측정하였다. 측정된 높이 행렬은 다음과 같다.

$$H = \begin{bmatrix} 12 & 14 & 16 & 15 & 13 \\ 13 & 18 & 22 & 20 & 16 \\ 15 & 23 & 31 & 25 & 18 \\ 14 & 20 & 26 & 24 & 17 \\ 11 & 15 & 18 & 16 & 14 \end{bmatrix}.$$

측정 범위는 $x = 1, 2, 3, 4, 5$, $y = 1, 2, 3, 4, 5$ 이다. `interp2`를 이용하여 더 촘촘한 지형 surface를 생성하시오.

- (a) `meshgrid(1:5,1:5)`를 사용하여 원래 격자 `X`, `Y`를 생성하시오.
- (b) 높이 행렬 `H`를 생성하시오.
- (c) $1:0.1:5$ 간격으로 촘촘한 격자 `Xq`, `Yq`를 생성하시오.
- (d) `interp2`와 `'linear'` 옵션을 사용하여 촘촘한 높이 행렬 `HqLinear`를 생성하시오.
- (e) `interp2`와 `'spline'` 옵션을 사용하여 촘촘한 높이 행렬 `HqSpline`을 생성하시오.
- (f) `tildeLayout(1,3)`을 사용하여 원래 지형, 선형보간 지형, 스플라인 보간 지형을 한 figure에 비교하시오.

(g) 각 tile에는 `surf`, `shading interp`, `colorbar`, 축 label, 제목을 모두 포함하시오.

10. 한 실험에서 시간에 따른 농도 변화를 측정했지만, 측정 간격이 일정하지 않다. 관측 시각과 농도는 다음과 같다.

Time	0	1.2	2.8	4.5	7.0	8.1	9.7	12.0
Concentration	5.0	6.4	9.1	8.6	6.8	5.9	5.2	4.8

일정한 시간 간격의 데이터로 변환한 뒤, 변화가 가장 급격한 구간을 찾으시오.

- 관측 시각을 `tObs`, 관측 농도를 `cObs`에 저장하시오.
 - `tq = 0:0.1:12`를 생성하시오.
 - 선형보간을 이용하여 `cqLinear`를 생성하시오.
 - 스플라인 보간을 이용하여 `cqSpline`을 생성하시오.
 - 선형보간 결과에 대해 인접한 농도 차이 `deltaC = diff(cqLinear)`를 계산하시오.
 - `abs(deltaC)`가 가장 큰 위치의 index를 `idxMaxChange`에 저장하시오.
 - `idxMaxChange`에 해당하는 시간 구간 `[tq(idxMaxChange), tq(idxMaxChange + 1)]`을 `changeInterval`에 저장하시오.
 - 관측 데이터, 선형보간 결과, 스플라인 보간 결과를 하나의 그래프에 나타내고, 변화가 가장 큰 구간의 두 끝점을 marker로 표시하시오.
11. 다음 함수는 감쇠 진동과 국소적 요동을 함께 갖는 신호이다.

$$y(t) = e^{-0.04t} \sin(2t) + 0.3e^{-0.01t} \cos(7t).$$

이 신호를 고해상도 그래프로 나타내고, 신호의 주요 구조를 시각적으로 강조하시오.

- `t = linspace(0,80,12000)`을 생성하시오.
 - 함수값을 `y`에 저장하시오.
 - `figure('Color','w')`를 사용하여 흰 배경의 figure를 생성하시오.
 - `plot`을 이용하여 `y`를 선 두께 1.3 이상으로 나타내시오.
 - `ylime(0)`을 추가하여 기준선을 표시하시오.
 - x축 label은 'Time', y축 label은 'Signal Amplitude', 제목은 'High-Resolution Damped Oscillatory Signal'로 설정하시오.
 - `xlim([0 80])`, `grid on`, `box on`을 적용하시오.
 - 축 글꼴 크기를 12 이상으로 설정하고 축 선 두께를 1.2 이상으로 설정하시오.
 - 최종 figure를 `damped_signal.png`라는 이름으로 300 dpi PNG 파일로 저장하시오.
12. 다음 2변수 함수는 중심부에서 진동이 강하고 바깥쪽으로 갈수록 감쇠하는 surface이다.

$$Z = e^{-0.06(X^2+Y^2)} \sin(4X) \cos(3Y) + 0.2 \sin(XY).$$

고해상도 surface 그래프로 시각화하시오.

- (a) x, y 를 각각 $[-5, 5]$ 구간에서 600개의 점으로 생성하시오.
- (b) `meshgrid`를 이용하여 X, Y 를 생성하시오.
- (c) 원소별 연산을 사용하여 Z 를 계산하시오.
- (d) `surf(X,Y,Z)`를 이용하여 surface 그래프를 그리시오.
- (e) `shading interp`, `colormap(jet)`, `colorbar`를 적용하시오.
- (f) x축 label, y축 label, z축 label을 각각 'X', 'Y', 'Z'로 지정하시오.
- (g) 제목은 'Detailed Surface Plot with Oscillation and Damping'으로 지정하시오.
- (h) `view(45,30)`, `camlight headlight`, `lighting phong`을 적용하시오.
- (i) `max(Z(:))`, `min(Z(:))`을 이용하여 최대값과 최소값을 각각 `maxZ`, `minZ`에 저장하시오.
- (j) 최대값과 최소값이 발생하는 위치의 (x, y, z) 좌표를 각각 `maxPoint`, `minPoint`에 저장하시오.

13. 다음 함수의 등고선 구조를 고해상도 filled contour로 시각화하시오.

$$Z = \frac{\cos(2X) \sin(3Y)}{1 + 0.1(X^2 + Y^2)}.$$

이 함수는 여러 개의 봉우리와 골짜기를 가지며, 중심에서 멀어질수록 진폭이 완만하게 감소한다.

- (a) x, y 를 각각 $[-8, 8]$ 구간에서 800개의 점으로 생성하시오.
- (b) `meshgrid`를 이용하여 X, Y 를 생성하시오.
- (c) Z 를 계산하시오.
- (d) `contourf(X,Y,Z,70,'LineColor','none')`을 이용하여 filled contour를 그리시오.
- (e) `colormap(parula)`, `colorbar`, `axis equal`, `grid on`, `box on`을 적용하시오.
- (f) x축 label은 'X coordinate', y축 label은 'Y coordinate'로 지정하시오.
- (g) 제목은 'High-Resolution Filled Contour of a Damped Oscillatory Field'로 지정하시오.
- (h) 축 글꼴 크기를 12 이상으로 설정하고, 축 선 두께를 1.2 이상으로 설정하시오.

14. 다음 세 가지 곡면을 하나의 figure에서 비교하시오.

$$Z_1 = \sin(X) \cos(Y), \quad Z_2 = e^{-0.08(X^2+Y^2)}, \quad Z_3 = \sin(X^2 + Y^2).$$

세 곡면은 각각 주기적 진동, 중심부 집중, 원형 파동 구조를 나타낸다.

- (a) x, y 를 각각 $[-4, 4]$ 구간에서 300개의 점으로 생성하시오.
- (b) `meshgrid`를 이용하여 X, Y 를 생성하시오.
- (c) Z_1, Z_2, Z_3 를 각각 계산하시오.
- (d) `tiledlayout(1,3)`을 사용하여 1행 3열 레이아웃을 생성하시오.
- (e) 각 tile에 `surf`를 이용하여 Z_1, Z_2, Z_3 를 각각 나타내시오.

- (f) 각 그래프에 `shading interp`, `colorbar`, x축 label, y축 label, z축 label, 제목을 포함 하시오.
- (g) 세 그래프 모두 `view(45,30)`을 적용하시오.
- (h) 전체 제목은 'Comparison of Three Surface Structures'로 설정하시오.

15. 매개변수 u, v 를 이용하여 비틀린 리본 형태의 3차원 곡면을 생성하시오.

$$X = (2 + 0.4 \cos v) \cos u, \quad Y = (2 + 0.4 \cos v) \sin u, \quad Z = 0.25u + 0.4 \sin v.$$

단, $0 \leq u \leq 8\pi$, $-\pi \leq v \leq \pi$ 이다.

- (a) `u = linspace(0,8*pi,500)`, `v = linspace(-pi,pi,180)`을 생성하시오.
- (b) `meshgrid`를 이용하여 `U, V`를 생성하시오.
- (c) 위 식을 이용하여 `X, Y, Z`를 계산하시오.
- (d) `surf(X,Y,Z)`를 이용하여 곡면을 그리시오.
- (e) `shading interp`, `colormap(parula)`, `colorbar`를 적용하시오.
- (f) x축 label, y축 label, z축 label을 각각 'X', 'Y', 'Z'로 지정하시오.
- (g) 제목은 'Twisted Parametric Ribbon Surface'로 지정하시오.
- (h) `axis equal`, `axis tight`, `grid on`, `view(45,35)`를 적용하시오.
- (i) 최종 figure를 `twisted.ribbon.png`라는 이름으로 300 dpi PNG 파일로 저장하시오.

16. 하나의 figure에 2차원 시계열 그래프, 3차원 surface 그래프, filled contour 그래프를 함께 배치하여 종합 dashboard를 구성하시오. 사용 데이터는 다음과 같다.

$$s(t) = \sin(0.5t) + 0.4 \sin(3t) + 0.15\epsilon, \quad Z = e^{-0.05(X^2+Y^2)} \cos(3X) \sin(2Y).$$

- (a) 난수 결과가 재현되도록 `rng(3)`을 설정하시오.
- (b) `t = linspace(0,60,6000)`을 생성하고, 정규분포 잡음을 포함한 `signal`을 생성하시오.
- (c) `movmean`을 사용하여 window size 200의 이동평균 `trend`를 생성하시오.
- (d) x, y 를 각각 $[-5, 5]$ 구간에서 300개의 점으로 생성하고, `meshgrid`로 `X, Y`를 생성하시오.
- (e) `Z`를 계산하시오.
- (f) `tiledlayout(2,2)`를 사용하여 dashboard를 생성하시오.
- (g) 첫 번째 tile에는 원본 `signal`과 이동평균 `trend`를 함께 나타내시오.
- (h) 두 번째 tile에는 `surf(X,Y,Z)`를 이용하여 surface를 나타내시오.
- (i) 세 번째 tile에는 `contourf(X,Y,Z,50,'LineColor','none')`을 이용하여 filled contour를 나타내시오.
- (j) 각 tile에 제목, 축 label, grid 또는 `colorbar`를 적절히 추가하시오.
- (k) 전체 제목은 'Integrated Visualization Dashboard'로 설정하시오.
- (l) 최종 figure를 `visualization_dashboard.png`라는 이름으로 300 dpi PNG 파일로 저장 하시오.

Department of Artificial Intelligence and Information Technology (Jin Hee Yoon)
Sejong University